

COMPSCI 589 Final Project - Spring 2026

Due **May 8**, 11:55 pm Eastern Time

Introduction

- Each group member was responsible for evaluating the performance of at least one unique algorithm implemented during the semester on each dataset
- The algorithms tested in this project were chosen from the pool of: k -NN, Decision Trees, multinomial Naive Bayes, Random Forests, and Neural Networks
- All work was done in the shared github repository https://github.com/19851xw/589_final_project

-
- **Katherine** trained the neural networks algorithm on all four datasets. She tested three hyperparameters: the architecture, the λ (regularization) parameter, and the α (learning rate) parameter.
 - **Tiffany** implemented KNN and Random Forest. She evaluated the hyperparameters k for kNN and n_{tree} for random forest.
 - **Xing-Wei** implemented Decision Trees. She evaluated two hyperparameters: max depth and minimum split size.

Experiments and Analyses

1 The Hand-Written Digits Recognition Dataset

1.1 Neural Networks Algorithm

Note that $k = 5$ was used for all cross-validation training with the neural network implementation particularly. This is due to the fact that $k = 10$ was computationally costly, and we found a smaller k value actually reduced the variance in model performance and was thus preferable.

We tested the neural network on the digits dataset because it is a multi-class classification problem with non-linear patterns in higher-dimensional data, which neural networks are able to learn. The hyperparameters optimized included the architecture (layer sizes), regularization values λ , and learning rates α .

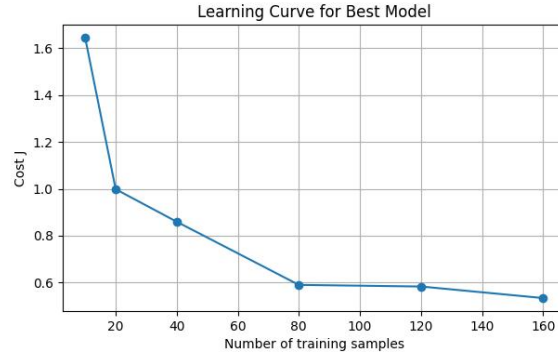
The following is a table of the performance, based on the accuracy and f1 score, of the neural network algorithm depending on the three hyperparameters.

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
(16,)	[64, 16, 10]	0.100000	0.100000	0.936010	0.003352	0.936004	0.003193
(16,)	[64, 16, 10]	0.010000	0.100000	0.928186	0.015246	0.928047	0.014974
(16,)	[64, 16, 10]	1.000000	0.100000	0.927031	0.014071	0.926733	0.014543
(16,)	[64, 16, 10]	0.001000	0.100000	0.926529	0.008509	0.926113	0.008459
(8,)	[64, 8, 10]	0.100000	0.100000	0.894838	0.013742	0.893430	0.013881
(8,)	[64, 8, 10]	0.010000	0.100000	0.893196	0.013038	0.890354	0.016084
(8,)	[64, 8, 10]	0.001000	0.100000	0.890460	0.011603	0.888083	0.014267
(16,)	[64, 16, 10]	0.001000	0.050000	0.890339	0.010015	0.887543	0.011267
(16,)	[64, 16, 10]	0.010000	0.050000	0.888567	0.020419	0.887115	0.021405
(16,)	[64, 16, 10]	0.100000	0.050000	0.883701	0.011069	0.881143	0.011120
(8,)	[64, 8, 10]	1.000000	0.100000	0.884751	0.014395	0.879637	0.018089
(16,)	[64, 16, 10]	1.000000	0.050000	0.878695	0.013204	0.873405	0.016985
(32, 16)	[64, 32, 16, 10]	0.001000	0.100000	0.827440	0.015375	0.819822	0.017508
(8,)	[64, 8, 10]	0.010000	0.050000	0.825702	0.023511	0.819201	0.023182
(32, 16)	[64, 32, 16, 10]	0.100000	0.100000	0.827006	0.008187	0.818914	0.008585
(8,)	[64, 8, 10]	0.001000	0.050000	0.830164	0.017450	0.817406	0.022305
(8,)	[64, 8, 10]	0.100000	0.050000	0.827516	0.032324	0.817083	0.042383
(32, 16)	[64, 32, 16, 10]	0.010000	0.100000	0.812512	0.012730	0.795429	0.012950
(8,)	[64, 8, 10]	1.000000	0.050000	0.784495	0.021175	0.765704	0.030110
(4,)	[64, 4, 10]	0.001000	0.100000	0.748785	0.034874	0.718616	0.049695
(4,)	[64, 4, 10]	0.010000	0.100000	0.715665	0.029049	0.684812	0.044523
(4,)	[64, 4, 10]	0.100000	0.100000	0.709086	0.038627	0.676275	0.050608
(32, 16)	[64, 32, 16, 10]	1.000000	0.100000	0.716524	0.047766	0.670612	0.060647
(4,)	[64, 4, 10]	1.000000	0.100000	0.698524	0.036531	0.656380	0.051966
(16, 8)	[64, 16, 8, 10]	0.001000	0.100000	0.684171	0.051285	0.653117	0.062733
(16, 8)	[64, 16, 8, 10]	0.010000	0.100000	0.680254	0.050111	0.644000	0.051728
(32, 16)	[64, 32, 16, 10]	0.001000	0.050000	0.673755	0.042180	0.636472	0.052423
(32, 16)	[64, 32, 16, 10]	0.100000	0.050000	0.669080	0.031326	0.632359	0.038613
(16,)	[64, 16, 10]	0.100000	0.010000	0.639125	0.043994	0.608531	0.052206
(4,)	[64, 4, 10]	0.001000	0.050000	0.651545	0.043477	0.603943	0.060162
(16,)	[64, 16, 10]	0.010000	0.010000	0.630338	0.051082	0.594015	0.065511
(16, 8)	[64, 16, 8, 10]	0.100000	0.100000	0.646200	0.082509	0.592538	0.097829
(16,)	[64, 16, 10]	0.001000	0.010000	0.611189	0.056685	0.584032	0.059974
(32, 16)	[64, 32, 16, 10]	1.000000	0.050000	0.624559	0.064782	0.576131	0.071108
(32, 16)	[64, 32, 16, 10]	0.010000	0.050000	0.616655	0.014354	0.568440	0.029587
(4,)	[64, 4, 10]	1.000000	0.050000	0.621424	0.053115	0.567201	0.075218
(4,)	[64, 4, 10]	0.010000	0.050000	0.616923	0.033609	0.562805	0.034687
(4,)	[64, 4, 10]	0.100000	0.050000	0.609148	0.037702	0.556908	0.046996

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
(16,)	[64, 16, 10]	1.000000	0.010000	0.586064	0.036527	0.549366	0.045586
(8,)	[64, 8, 10]	0.001000	0.010000	0.518192	0.064763	0.481672	0.080454
(16, 8)	[64, 16, 8, 10]	0.001000	0.050000	0.505000	0.047957	0.466305	0.064571
(16, 8)	[64, 16, 8, 10]	0.100000	0.050000	0.515970	0.062050	0.453043	0.071009
(8,)	[64, 8, 10]	1.000000	0.010000	0.477887	0.052700	0.432679	0.066033
(8,)	[64, 8, 10]	0.010000	0.010000	0.440336	0.021814	0.385214	0.038638
(8,)	[64, 8, 10]	0.100000	0.010000	0.432680	0.074723	0.375216	0.086287
(16, 8)	[64, 16, 8, 10]	0.010000	0.050000	0.454082	0.061510	0.374900	0.074638
(16, 8)	[64, 16, 8, 10]	1.000000	0.050000	0.455649	0.078360	0.358436	0.082761
(16, 8)	[64, 16, 8, 10]	1.000000	0.100000	0.442136	0.054345	0.347818	0.066084
(32, 16)	[64, 32, 16, 10]	0.001000	0.010000	0.316155	0.052282	0.277157	0.049574
(32, 16, 8)	[64, 32, 16, 8, 10]	0.010000	0.100000	0.338410	0.057837	0.260971	0.044447
(32, 16)	[64, 32, 16, 10]	0.100000	0.010000	0.261184	0.025400	0.220344	0.021318
(32, 16)	[64, 32, 16, 10]	0.010000	0.010000	0.286987	0.054219	0.209054	0.050013
(4,)	[64, 4, 10]	1.000000	0.010000	0.280106	0.060204	0.189312	0.057634
(4,)	[64, 4, 10]	0.001000	0.010000	0.278834	0.058559	0.183091	0.054619
(32, 16)	[64, 32, 16, 10]	1.000000	0.010000	0.220673	0.043832	0.164759	0.048841
(32, 16, 8)	[64, 32, 16, 8, 10]	0.100000	0.100000	0.227292	0.070533	0.160040	0.068864
(32, 16, 8)	[64, 32, 16, 8, 10]	0.001000	0.100000	0.251414	0.058767	0.157138	0.057873
(4,)	[64, 4, 10]	0.010000	0.010000	0.230382	0.077220	0.148625	0.067600
(4,)	[64, 4, 10]	0.100000	0.010000	0.237988	0.059281	0.143756	0.063367
(16, 8)	[64, 16, 8, 10]	0.100000	0.010000	0.222142	0.057739	0.133770	0.051809
(16, 8)	[64, 16, 8, 10]	0.001000	0.010000	0.182262	0.053205	0.110918	0.058702
(16, 8)	[64, 16, 8, 10]	1.000000	0.010000	0.192482	0.015338	0.109596	0.011034
(16,)	[64, 16, 10]	0.001000	0.001000	0.160072	0.038361	0.098771	0.049695
(32, 16, 8)	[64, 32, 16, 8, 10]	0.010000	0.050000	0.184528	0.053773	0.093284	0.057699
(16, 8)	[64, 16, 8, 10]	0.010000	0.010000	0.149165	0.023554	0.086936	0.031117
(16,)	[64, 16, 10]	1.000000	0.001000	0.131191	0.019411	0.066803	0.019126
(32, 16, 8)	[64, 32, 16, 8, 10]	0.100000	0.050000	0.151919	0.047991	0.065793	0.036933
(8,)	[64, 8, 10]	1.000000	0.001000	0.118087	0.027549	0.064932	0.022850
(32, 16, 8)	[64, 32, 16, 8, 10]	0.001000	0.050000	0.151368	0.049614	0.063872	0.036731
(8,)	[64, 8, 10]	0.010000	0.001000	0.121862	0.032156	0.058619	0.037514
(32, 16, 8)	[64, 32, 16, 8, 10]	1.000000	0.050000	0.149167	0.064202	0.055642	0.053056
(8,)	[64, 8, 10]	0.001000	0.001000	0.109036	0.029176	0.048114	0.021917
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.100000	0.050000	0.126024	0.034909	0.045723	0.022648
(16,)	[64, 16, 10]	0.100000	0.001000	0.112612	0.025945	0.045323	0.024360
(4,)	[64, 4, 10]	1.000000	0.001000	0.107912	0.017228	0.044427	0.014491
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.001000	0.100000	0.131391	0.033698	0.038900	0.020546
(32, 16, 8)	[64, 32, 16, 8, 10]	0.001000	0.010000	0.107890	0.022343	0.036948	0.020578
(8,)	[64, 8, 10]	0.100000	0.001000	0.096791	0.012696	0.036349	0.022264
(16,)	[64, 16, 10]	0.010000	0.001000	0.080126	0.020082	0.036322	0.009351
(4,)	[64, 4, 10]	0.001000	0.001000	0.114034	0.024161	0.035294	0.015053
(32, 16)	[64, 32, 16, 10]	1.000000	0.001000	0.109682	0.013319	0.034734	0.021411
(32, 16, 8)	[64, 32, 16, 8, 10]	0.010000	0.010000	0.098041	0.038728	0.033464	0.019659
(4,)	[64, 4, 10]	0.010000	0.001000	0.082974	0.023527	0.032210	0.013778
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.010000	0.100000	0.115167	0.017844	0.031755	0.016726
(32, 16)	[64, 32, 16, 10]	0.010000	0.001000	0.109038	0.009781	0.031646	0.015099
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.010000	0.050000	0.114978	0.031879	0.031011	0.018672
(32, 16)	[64, 32, 16, 10]	0.100000	0.001000	0.094121	0.030868	0.030121	0.010352
(16, 8)	[64, 16, 8, 10]	1.000000	0.001000	0.105756	0.005422	0.028300	0.008169
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.100000	0.010000	0.104594	0.005166	0.026886	0.010846
(32, 16, 8)	[64, 32, 16, 8, 10]	1.000000	0.010000	0.111865	0.024908	0.026449	0.016678
(16, 8)	[64, 16, 8, 10]	0.100000	0.001000	0.104635	0.006947	0.026445	0.008238
(32, 16, 8)	[64, 32, 16, 8, 10]	0.010000	0.001000	0.114723	0.026680	0.026156	0.015491

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
(32, 16, 8)	[64, 32, 16, 8, 10]	0.100000	0.010000	0.093875	0.034404	0.025944	0.017285
(32, 16)	[64, 32, 16, 10]	0.001000	0.001000	0.104626	0.006070	0.025064	0.007841
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.001000	0.050000	0.103527	0.004501	0.023816	0.006950
(16, 8)	[64, 16, 8, 10]	0.001000	0.001000	0.095094	0.009591	0.022273	0.006775
(32, 16, 8)	[64, 32, 16, 8, 10]	1.000000	0.100000	0.102922	0.004900	0.022078	0.007631
(4,)	[64, 4, 10]	0.100000	0.001000	0.062750	0.036817	0.021216	0.009735
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.010000	0.001000	0.096874	0.007405	0.019645	0.002779
(32, 16, 8)	[64, 32, 16, 8, 10]	0.001000	0.001000	0.101285	0.001408	0.019221	0.001809
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.001000	0.010000	0.092475	0.014706	0.018912	0.000970
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.100000	0.100000	0.101277	0.000665	0.018460	0.000182
(16, 8)	[64, 16, 8, 10]	0.010000	0.001000	0.100172	0.001807	0.018414	0.000065
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.001000	0.001000	0.101277	0.000665	0.018393	0.000110
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	1.000000	0.100000	0.101277	0.000665	0.018393	0.000110
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	1.000000	0.010000	0.095711	0.010279	0.018310	0.000127
(32, 16, 8)	[64, 32, 16, 8, 10]	1.000000	0.001000	0.100732	0.001477	0.018302	0.000244
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	1.000000	0.001000	0.100717	0.001479	0.018300	0.000244
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.010000	0.010000	0.099612	0.000840	0.018118	0.000139
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	1.000000	0.050000	0.092933	0.014461	0.017121	0.002177
(64, 32, 16, 8)	[64, 64, 32, 16, 8, 10]	0.100000	0.001000	0.092433	0.013168	0.017112	0.001816
(32, 16, 8)	[64, 32, 16, 8, 10]	0.100000	0.001000	0.086802	0.026749	0.016655	0.003115

The best result is around 93.6% accuracy and 93.6% F1 with a moderate-size network [64, 16, 10], using $\alpha = 0.1$ and $\lambda = 0.1$. That suggests the task is complex enough to need a hidden layer, but not so complex that a very deep network is necessary. Below is a learning curve showing the value of J as a function of the number of training instances presented to the above best network. As evidenced, the cost decreases as a function of the number of training instances, which is as expected and proves that the neural network algorithm is able to learn the digits dataset.



1.2 KNN

k	Accuracy	F1 Score
1	0.988571	0.989391
3	0.986857	0.987597
5	0.986286	0.987128
7	0.986286	0.987206
9	0.982857	0.984203
11	0.980571	0.982039
13	0.980000	0.980917
15	0.980571	0.982108
17	0.977143	0.978787
19	0.976000	0.977379
21	0.974857	0.976219
23	0.972571	0.974215
25	0.969714	0.971757
27	0.966286	0.968297
29	0.968571	0.970363
31	0.967429	0.968898
33	0.961714	0.963689
35	0.964571	0.966932
37	0.963429	0.965532
39	0.963429	0.965328
41	0.960571	0.962805
43	0.961143	0.962758
45	0.959429	0.961605
47	0.956571	0.959014
49	0.957714	0.960383
51	0.955429	0.958529



Figure 1: Handwritten Digits Accuracy Distribution

As k increases, the general trend for the performance is that it decreases. This is expected, as too large k values can lead to too generalized performance. Although, the overall performance is still generally high with the lowest accuracy and F1 score being around 0.96. The best configuration for this model based on this graph is at

$k = 1$. We chose KNN for this dataset because the same numbers would be in similar places in 64-dimensional space, so we thought this algorithm would perform well (which it did!).

1.3 Random Forest

ntree	1	5	10	20	30	40	50
Accuracy	0.968223	0.969386	0.993427	1.0	0.996774	0.997059	1.0
Precision	0.954737	1.000000	1.000000	1.0	1.000000	1.000000	1.0
Recall	0.980000	0.934332	0.985000	1.0	0.993333	0.994118	1.0
F1 Score	0.962738	0.965294	0.992204	1.0	0.996552	0.996970	1.0

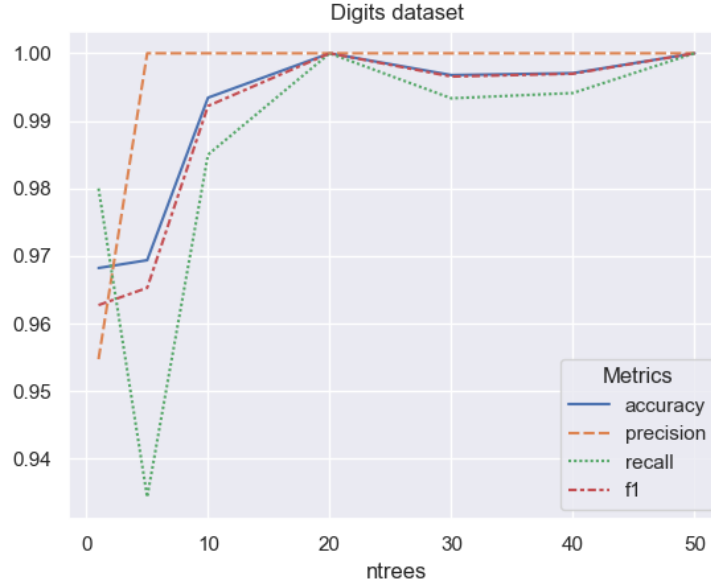


Figure 2: Digits Accuracy and F1 Score Distribution

Technically the highest accuracy and f1 occurs at $\text{ntree}=20$ and $\text{ntree}=50$ because the accuracy and f1 are both 1 with those values, but it is likely that the model is overfit. Because of this, it may be better to use a value of $\text{ntree}=5$ even though the performance is technically a little lower at an accuracy of 0.969386 and an f1 score of 0.965294. We chose random forest for digits because this dataset has a lot of attributes (64) so in order for a decision tree to perform as well, it would have to be very complex. In contrast, with a random forest each tree can be less complex and still perform well.

2 The Oxford Parkinson's Disease Detection Dataset

2.1 Decision Tree Algorithm

We chose to use decision trees for this dataset because decision trees are best at finding precise boundaries when there are only numerical features. This is the case for the Parkinson's dataset, which has 22 numerical features and no categorical features. We use the hyperparameters of max depth and minimum split size to control the complexity of the decision tree. Below we tested 7 different tree depths and 4 different minimum split sizes.

Tuning for dataset with 195 rows and 22 features.

Testing Depths: [2, 3, 5, 7, 9, 11, 13]

Testing Splits: [9, 19, 29, 39]

Depth	Split	Test Acc	Test F1
2	9	0.8503	0.8065
2	19	0.8117	0.7631
2	29	0.8454	0.7922
2	39	0.8625	0.7836
3	9	0.8681	0.8278
3	19	0.8312	0.7890
3	29	0.8309	0.7553
3	39	0.8620	0.7975
5	9	0.8575	0.8157
5	19	0.8556	0.8171
5	29	0.8362	0.7784
5	39	0.8611	0.7906
7	9	0.8456	0.8050
7	19	0.8211	0.7456
7	29	0.8312	0.7815
7	39	0.8617	0.7912
9	9	0.8323	0.7762
9	19	0.8515	0.8086
9	29	0.8534	0.8038
9	39	0.8512	0.7767
11	9	0.8562	0.7975
11	19	0.8220	0.7627
11	29	0.8273	0.7595
11	39	0.8606	0.7840
13	9	0.8628	0.8177
13	19	0.8361	0.7720
13	29	0.8348	0.7692
13	39	0.8495	0.7635

=====

BEST CONFIGURATION:

Max Depth: 3

Min Split: 9

F1 Score: 0.8278

=====

Final Results:

Training Accuracy: 0.9345

Training F1 Score: 0.9165

Testing Accuracy: 0.8508

Testing F1 Score: 0.8101

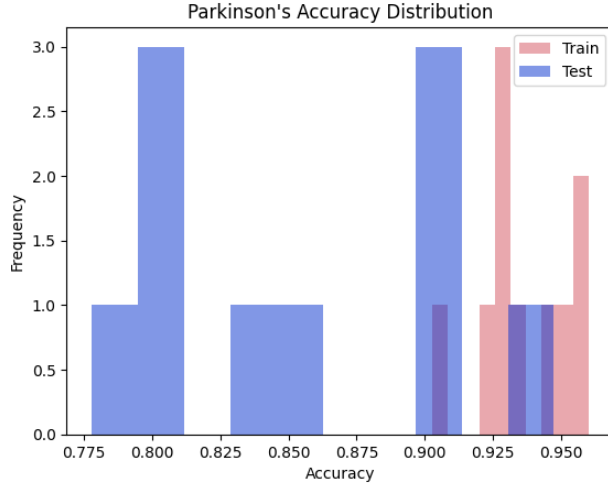


Figure 3: Parkinson's Accuracy Distribution

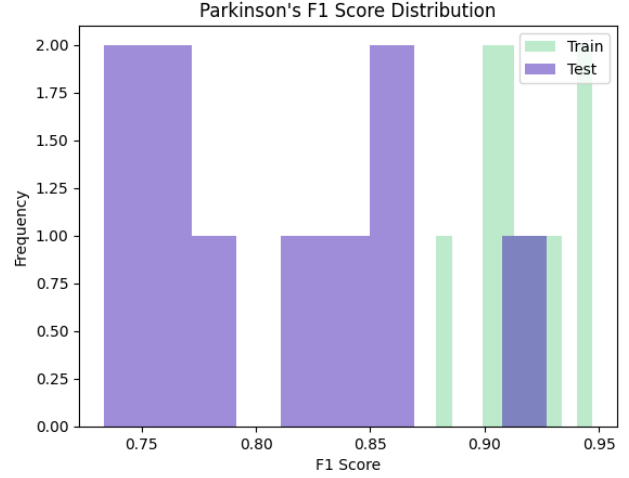


Figure 4: Parkinson's F1 Score Distribution

There is high variance on the testing scores, which may be due to the small size of the dataset making the model very sensitive to specific training data in each fold. The training scores are very concentrated around 0.93 which suggests high stability and low bias. There is a clear difference between the average training and testing scores, which suggest the model may be overfitting. Since the model is sensitive to specific training data, the model is less robust and may not generalize well to unseen data. Decision trees are well suited for numerical datasets, but when the sample size is small, the splits can become too specific to a training set which leads to the variance seen in testing.

2.2 KNN

k	Accuracy	F1 Score
1	0.966667	0.978033
3	0.944444	0.963738
5	0.944444	0.964514
7	0.944444	0.965475
9	0.927778	0.954590
11	0.900000	0.937926
13	0.872222	0.922714
15	0.861111	0.915862
17	0.850000	0.910674
19	0.838889	0.903946
21	0.811111	0.890880
23	0.827778	0.899946
25	0.850000	0.912620
27	0.838889	0.906579
29	0.838889	0.906613
31	0.844444	0.909247
33	0.866667	0.922270
35	0.872222	0.924905
37	0.872222	0.924716
39	0.850000	0.912842
41	0.855556	0.915269
43	0.833333	0.903978
45	0.844444	0.909831
47	0.822222	0.897769
49	0.827778	0.900780
51	0.816667	0.895134



Figure 5: Parkinsons Accuracy Distribution

Again like the digits dataset, as k increases, the general performance decreases. However, the difference between the highest and the lowest performing k value is much larger this time. The best configuration for this model based on this graph is at $k = 1$. We chose KNN for this dataset because the algorithm performs well on both numerical data, and also this dataset is not that large so it would perform relatively fast.

2.3 Random Forest

ntree	1	5	10	20	30	40	50
Accuracy	0.805556	0.794444	0.816667	0.833333	0.850000	0.838889	0.850000
Precision	0.836293	0.843709	0.853481	0.853754	0.861730	0.854793	0.872728
Recall	0.935714	0.907143	0.928571	0.950000	0.964286	0.957143	0.950000
F1 Score	0.881610	0.872319	0.887543	0.898543	0.909364	0.902490	0.908122

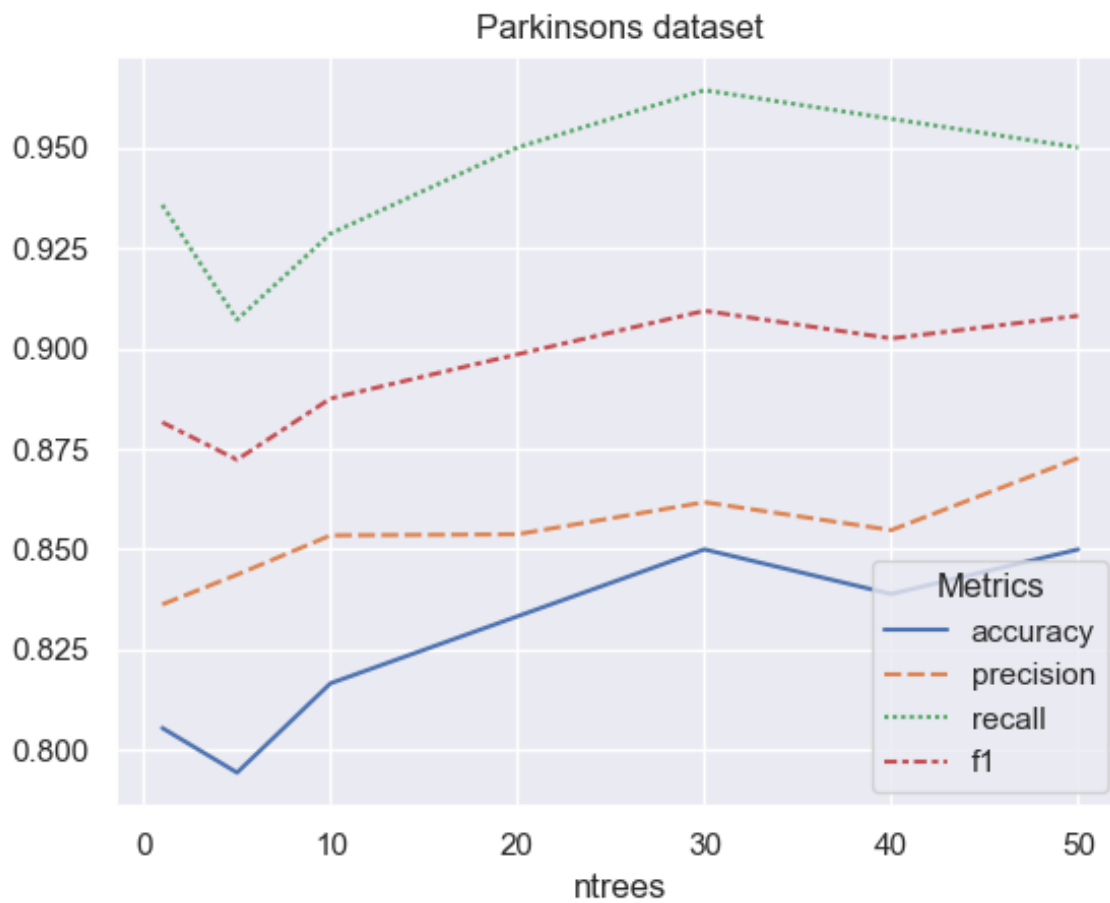


Figure 6: Parkinsons Accuracy and F1 Score Distribution

The best accuracy happens at ntree=30,50 at 85%. As ntree increases, the accuracy and F1 score generally increases with only a small dip by 2% at around 40 trees. We chose random forest because the dataset is very small: we didn't want a decision tree to overfit on the data, as the tend to do.

3 The Rice Grain Dataset

3.1 Neural Networks Algorithm

We tested the algorithm on the rice dataset because it has few features and a fairly large sample size, making it an optimal task for testing our network on learning a simpler problem. The hyperparameters optimized included the architecture (layer sizes), regularization values λ , and learning rates α .

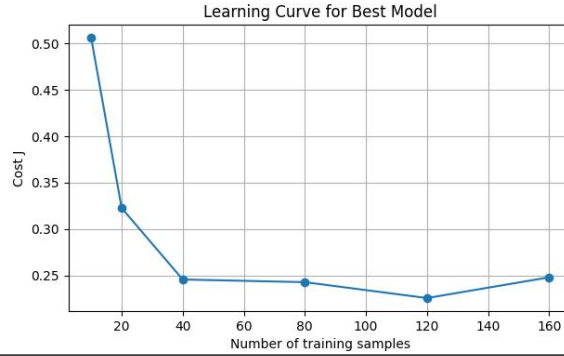
The following is a table of the performance, based on the accuracy and f1 score, of the neural network algorithm depending on the three hyperparameters.

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
((4,))	[7, 4, 1]	1.000000	0.100000	0.928871	0.001741	0.937713	0.001479
(8,)	[7, 8, 1]	0.100000	0.100000	0.928609	0.001964	0.937463	0.001891
(16,)	[7, 16, 1]	0.001000	0.100000	0.928609	0.003061	0.937442	0.002663
(8,)	[7, 8, 1]	0.010000	0.100000	0.928346	0.002702	0.937171	0.002269
(16,)	[7, 16, 1]	1.000000	0.100000	0.928084	0.002545	0.937004	0.002355
(16,)	[7, 16, 1]	0.010000	0.100000	0.928084	0.004345	0.936839	0.003752
(8,)	[7, 8, 1]	1.000000	0.100000	0.927559	0.003560	0.936539	0.003319
(8,)	[7, 8, 1]	0.001000	0.100000	0.927297	0.002946	0.936274	0.002627
(16,)	[7, 16, 1]	0.100000	0.100000	0.927297	0.002946	0.936209	0.002781
(16, 8)	[7, 16, 8, 1]	0.100000	0.100000	0.927297	0.001780	0.936178	0.001918
(16, 8)	[7, 16, 8, 1]	0.010000	0.100000	0.926772	0.003840	0.935913	0.003692
(32, 16)	[7, 32, 16, 1]	0.001000	0.100000	0.926772	0.001929	0.935833	0.002096
(16,)	[7, 16, 1]	0.001000	0.050000	0.926509	0.001856	0.935825	0.002009
(32, 16)	[7, 32, 16, 1]	0.010000	0.100000	0.926772	0.003257	0.935796	0.003237
(16,)	[7, 16, 1]	1.000000	0.050000	0.926247	0.003361	0.935793	0.003029
(16,)	[7, 16, 1]	0.010000	0.050000	0.926247	0.003560	0.935550	0.003358
(32, 16)	[7, 32, 16, 1]	1.000000	0.100000	0.925984	0.003767	0.935428	0.003497
(16, 8)	[7, 16, 8, 1]	0.001000	0.100000	0.926247	0.002545	0.935381	0.002382
(32, 16)	[7, 32, 16, 1]	0.100000	0.100000	0.926247	0.002802	0.935368	0.002844
(8,)	[7, 8, 1]	1.000000	0.050000	0.925722	0.002572	0.935120	0.002611
(4,)	[7, 4, 1]	0.100000	0.100000	0.925984	0.002572	0.934980	0.002560
(4,)	[7, 4, 1]	0.001000	0.050000	0.925459	0.003928	0.934936	0.003681
(8,)	[7, 8, 1]	0.100000	0.050000	0.925722	0.004591	0.934834	0.004478
(8,)	[7, 8, 1]	0.010000	0.050000	0.924934	0.002258	0.934325	0.002449
(4,)	[7, 4, 1]	0.010000	0.100000	0.925197	0.002753	0.934305	0.002648
(4,)	[7, 4, 1]	1.000000	0.050000	0.924934	0.004015	0.934268	0.003691
(16, 8)	[7, 16, 8, 1]	1.000000	0.100000	0.924934	0.003560	0.934258	0.003539
(4,)	[7, 4, 1]	0.001000	0.100000	0.924934	0.002802	0.934252	0.002447
(16,)	[7, 16, 1]	0.100000	0.050000	0.924672	0.003382	0.934174	0.003208
(32, 16)	[7, 32, 16, 1]	0.100000	0.050000	0.923360	0.004199	0.933364	0.004084
(4,)	[7, 4, 1]	0.100000	0.050000	0.923622	0.002258	0.933330	0.002029
(8,)	[7, 8, 1]	0.001000	0.050000	0.923885	0.002875	0.933287	0.002850
(16, 8)	[7, 16, 8, 1]	0.100000	0.050000	0.922572	0.003712	0.932879	0.003138
(8,)	[7, 8, 1]	0.010000	0.010000	0.920997	0.005951	0.932572	0.004861
(32, 16)	[7, 32, 16, 1]	0.010000	0.050000	0.922310	0.004868	0.932301	0.004577
(32, 16, 8)	[7, 32, 16, 8, 1]	0.010000	0.100000	0.920472	0.006874	0.931727	0.004592
(4,)	[7, 4, 1]	0.010000	0.050000	0.921522	0.004423	0.931587	0.003960
(32, 16)	[7, 32, 16, 1]	1.000000	0.050000	0.920735	0.003061	0.931547	0.002635
(16, 8)	[7, 16, 8, 1]	0.010000	0.050000	0.920997	0.002545	0.931447	0.002223
(32, 16, 8)	[7, 32, 16, 8, 1]	0.100000	0.100000	0.919948	0.001856	0.930331	0.002147
(32, 16)	[7, 32, 16, 1]	0.001000	0.050000	0.919423	0.005157	0.930329	0.004581
(16, 8)	[7, 16, 8, 1]	0.001000	0.050000	0.918373	0.008800	0.929816	0.007124
(16, 8)	[7, 16, 8, 1]	1.000000	0.050000	0.916798	0.007121	0.928683	0.005600
(16,)	[7, 16, 1]	0.010000	0.010000	0.915223	0.004199	0.928087	0.003287

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
(16,)	[7, 16, 1]	0.001000	0.010000	0.912861	0.004666	0.926091	0.003877
(16,)	[7, 16, 1]	1.000000	0.010000	0.912861	0.007724	0.925690	0.006526
(16,)	[7, 16, 1]	0.100000	0.010000	0.910499	0.010313	0.924214	0.008304
(8,)	[7, 8, 1]	0.001000	0.010000	0.902625	0.014552	0.918298	0.010501
(8,)	[7, 8, 1]	0.100000	0.010000	0.898425	0.005605	0.914774	0.003585
(4,)	[7, 4, 1]	0.010000	0.010000	0.897375	0.013598	0.914251	0.011200
(8,)	[7, 8, 1]	1.000000	0.010000	0.895013	0.014684	0.913485	0.009829
(4,)	[7, 4, 1]	0.100000	0.010000	0.892913	0.039323	0.912046	0.027703
(32, 16, 8)	[7, 32, 16, 8, 1]	0.001000	0.100000	0.849869	0.138948	0.889707	0.081035
(4,)	[7, 4, 1]	1.000000	0.010000	0.861942	0.049754	0.889365	0.033156
(4,)	[7, 4, 1]	0.001000	0.010000	0.854331	0.043056	0.886120	0.029145
(32, 16)	[7, 32, 16, 1]	0.010000	0.010000	0.703150	0.077476	0.796123	0.041932
(32, 16)	[7, 32, 16, 1]	1.000000	0.010000	0.695538	0.101181	0.793623	0.055403
(32, 16, 8)	[7, 32, 16, 8, 1]	1.000000	0.100000	0.683465	0.142356	0.790479	0.081706
(32, 16, 8)	[7, 32, 16, 8, 1]	0.010000	0.050000	0.659843	0.110031	0.775085	0.059707
(32, 16)	[7, 32, 16, 1]	0.001000	0.010000	0.637008	0.062551	0.760283	0.031589
(16,)	[7, 16, 1]	1.000000	0.001000	0.639108	0.139048	0.759762	0.085516
(32, 16)	[7, 32, 16, 1]	0.001000	0.001000	0.623097	0.078308	0.749741	0.043108
(16, 8)	[7, 16, 8, 1]	0.100000	0.010000	0.614173	0.052729	0.748762	0.026312
(16, 8)	[7, 16, 8, 1]	0.001000	0.010000	0.597375	0.027570	0.739974	0.013397
(32, 16)	[7, 32, 16, 1]	0.100000	0.010000	0.580315	0.016273	0.731747	0.007734
(16, 8)	[7, 16, 8, 1]	0.010000	0.010000	0.580052	0.013280	0.731597	0.006286
(32, 16, 8)	[7, 32, 16, 8, 1]	1.000000	0.050000	0.579003	0.013648	0.731109	0.006459
(32, 16, 8)	[7, 32, 16, 8, 1]	0.001000	0.050000	0.575853	0.007349	0.729601	0.003443
(32, 16, 8)	[7, 32, 16, 8, 1]	0.100000	0.050000	0.573491	0.002625	0.728490	0.001220
(16, 8)	[7, 16, 8, 1]	1.000000	0.010000	0.573228	0.002100	0.728367	0.000975
(32, 16, 8)	[7, 32, 16, 8, 1]	0.001000	0.010000	0.572178	0.000000	0.727880	0.000000
(32, 16, 8)	[7, 32, 16, 8, 1]	0.010000	0.010000	0.572178	0.000000	0.727880	0.000000
(32, 16, 8)	[7, 32, 16, 8, 1]	0.100000	0.010000	0.572178	0.000000	0.727880	0.000000
(32, 16, 8)	[7, 32, 16, 8, 1]	1.000000	0.010000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.001000	0.010000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.001000	0.050000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.001000	0.100000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.010000	0.010000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.010000	0.050000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.010000	0.100000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.100000	0.010000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.100000	0.050000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.100000	0.100000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	1.000000	0.010000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	1.000000	0.050000	0.572178	0.000000	0.727880	0.000000
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	1.000000	0.100000	0.572178	0.000000	0.727880	0.000000
(16,)	[7, 16, 1]	0.010000	0.001000	0.510236	0.163489	0.640448	0.189563
(8,)	[7, 8, 1]	0.001000	0.001000	0.587664	0.247858	0.632729	0.319038
(32, 16)	[7, 32, 16, 1]	0.010000	0.001000	0.613123	0.163899	0.623060	0.321372
(8,)	[7, 8, 1]	1.000000	0.001000	0.550919	0.082872	0.595471	0.281267
(16, 8)	[7, 16, 8, 1]	0.010000	0.001000	0.544357	0.055643	0.585940	0.283879
(16, 8)	[7, 16, 8, 1]	0.001000	0.001000	0.543307	0.057743	0.582304	0.291152
(32, 16, 8)	[7, 32, 16, 8, 1]	1.000000	0.001000	0.543307	0.057743	0.582304	0.291152
(4,)	[7, 4, 1]	1.000000	0.001000	0.606037	0.109378	0.580233	0.302614
(8,)	[7, 8, 1]	0.010000	0.001000	0.596063	0.126852	0.554502	0.285171
(4,)	[7, 4, 1]	0.010000	0.001000	0.511549	0.172871	0.530201	0.305849
(16,)	[7, 16, 1]	0.100000	0.001000	0.534646	0.137998	0.501061	0.316674
(32, 16)	[7, 32, 16, 1]	1.000000	0.001000	0.514436	0.070720	0.436728	0.356587

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
(32, 16, 8)	[7, 32, 16, 8, 1]	0.010000	0.001000	0.514436	0.070720	0.436728	0.356587
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.100000	0.001000	0.514436	0.070720	0.436728	0.356587
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	1.000000	0.001000	0.514436	0.070720	0.436728	0.356587
(16, 8)	[7, 16, 8, 1]	1.000000	0.001000	0.454331	0.111850	0.436185	0.275184
(16,)	[7, 16, 1]	0.001000	0.001000	0.518373	0.074228	0.435143	0.345196
(16, 8)	[7, 16, 8, 1]	0.100000	0.001000	0.495276	0.065246	0.420219	0.344428
(8,)	[7, 8, 1]	0.100000	0.001000	0.458005	0.124787	0.338180	0.250488
(32, 16, 8)	[7, 32, 16, 8, 1]	0.001000	0.001000	0.462467	0.099008	0.311083	0.342253
(32, 16)	[7, 32, 16, 1]	0.100000	0.001000	0.485564	0.070720	0.291152	0.356587
(32, 16, 8)	[7, 32, 16, 8, 1]	0.100000	0.001000	0.485564	0.070720	0.291152	0.356587
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.010000	0.001000	0.485564	0.070720	0.291152	0.356587
(4,)	[7, 4, 1]	0.100000	0.001000	0.491864	0.149961	0.229036	0.287047
(4,)	[7, 4, 1]	0.001000	0.001000	0.418898	0.152686	0.228141	0.285061
(64, 32, 16, 8)	[7, 64, 32, 16, 8, 1]	0.001000	0.001000	0.456693	0.057743	0.145576	0.291152

The best configuration was layer sizes of [7, 4, 1] with $\lambda = 1.0$ and $\alpha = 0.1$ with 92.9% accuracy and 93.8% F1. A noticeable pattern here is that the best models were the smallest ones, including [7, 4, 1], [7, 8, 1], and [7, 16, 1]. This suggests the classification boundary is relatively simple and that adding more complexity may just increase risks of overfitting or other training difficulties. Below is a learning curve showing the value of J as a function of the number of training instances presented to the above best network. As evidenced, the cost decreases as a function of the number of training instances, which is as expected and proves that the neural network algorithm is able to learn the rice dataset.



3.2 KNN

k	Accuracy	F1 Score
1	0.886614	0.867819
3	0.909974	0.894640
5	0.918898	0.904873
7	0.923885	0.910538
9	0.922047	0.908377
11	0.922835	0.909521
13	0.924409	0.911221
15	0.925984	0.913105
17	0.925984	0.912772
19	0.927034	0.914136
21	0.926509	0.913226
23	0.927034	0.914092
25	0.928084	0.915473
27	0.925984	0.912969
29	0.925459	0.912357
31	0.928346	0.915703
33	0.926509	0.913735
35	0.925984	0.912977
37	0.926509	0.913497
39	0.927559	0.915076
41	0.927034	0.914137
43	0.927559	0.914960
45	0.924934	0.911821
47	0.927559	0.914945
49	0.927559	0.914773
51	0.928084	0.915623

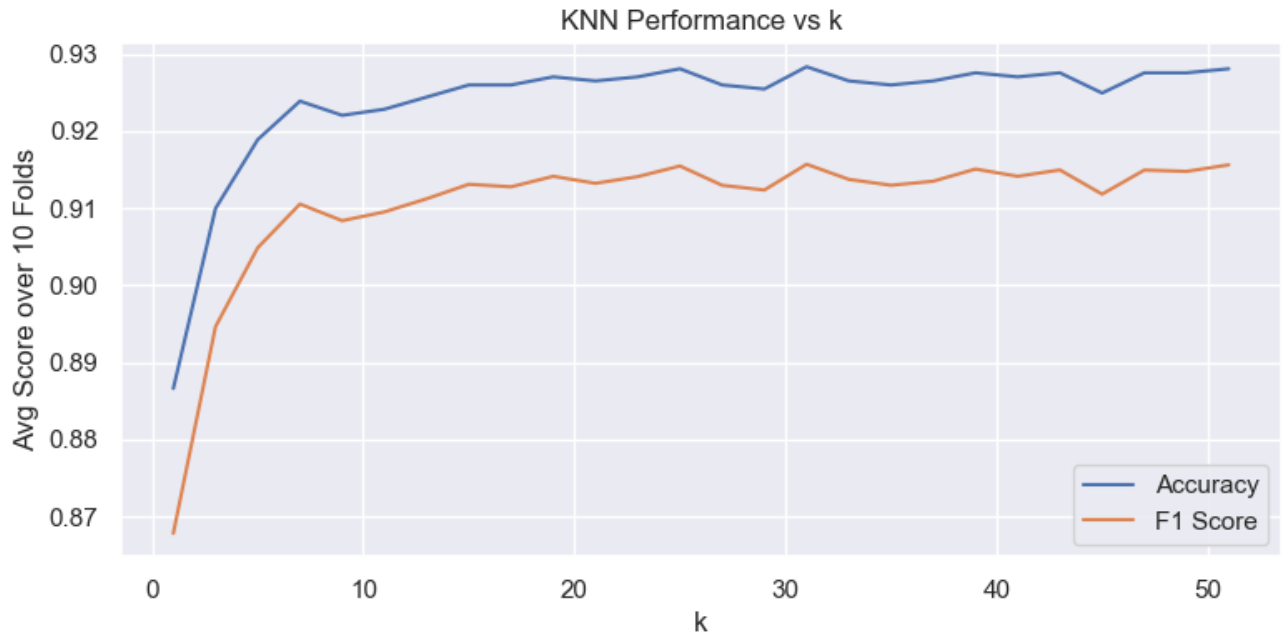


Figure 7: Rice Accuracy and F1 Score Distribution

After $k=1$, the performance gradually increases until it hits a plateau at around 92%. I would say the best configuration is at $k=7$ since it achieves basically the same performance as the higher k values but with a much

smaller k value and therefore much faster runtime. We chose KNN for this dataset because there are only a few numerical attributes, which may aid total runtime, and KNN performs well for numerical data.

3.3 Decision Trees Algorithm

We chose to test decision trees on the rice dataset because it has a moderate number of features (7) and a relatively large number of samples (3810), which allows us to test two different hyperparameters; tree depths and minimum split sizes. This is shown in the tuning results below, where we tested 6 different tree depths and 4 different minimum split sizes. us to test two different hyperparameters; tree depths and minimum split sizes. This is shown in the tuning results below, where we tested 6 different tree depths and 4 different minimum split sizes.

Tuning for dataset with 3810 rows and 7 features.

Testing Depths: [2, 3, 5, 7, 9, 11]

Testing Splits: [190, 381, 571, 762]

Depth	Split	Test Acc	Test F1
2	190	0.9215	0.9192
2	381	0.9228	0.9205
2	571	0.9226	0.9202
2	762	0.9239	0.9215
3	190	0.9234	0.9210
3	381	0.9213	0.9189
3	571	0.9202	0.9179
3	762	0.9239	0.9215
5	190	0.9213	0.9189
5	381	0.9234	0.9210
5	571	0.9226	0.9202
5	762	0.9239	0.9215
7	190	0.9223	0.9199
7	381	0.9226	0.9203
7	571	0.9215	0.9192
7	762	0.9239	0.9215
9	190	0.9234	0.9210
9	381	0.9220	0.9197
9	571	0.9239	0.9215
9	762	0.9239	0.9215
11	190	0.9223	0.9199
11	381	0.9220	0.9197
11	571	0.9218	0.9194
11	762	0.9239	0.9215

```
=====
BEST CONFIGURATION:
Max Depth: 11
Min Split: 762
F1 Score: 0.9215
=====
```

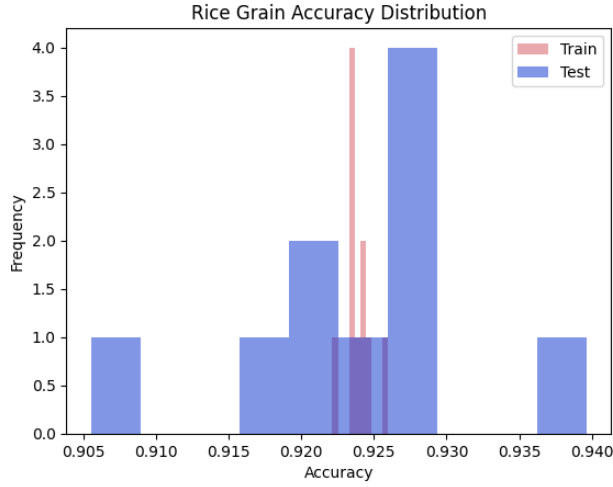


Figure 8: Rice Accuracy Distribution

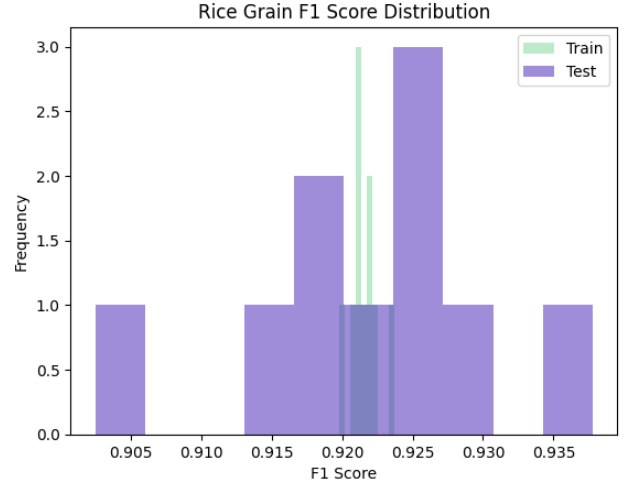


Figure 9: Rice F1 Score Distribution

Final Results:

Training Accuracy: 0.9239
 Training F1 Score: 0.9216
 Testing Accuracy: 0.9239
 Testing F1 Score: 0.9215

Final Results:

Training Accuracy: 0.9239
 Training F1 Score: 0.9216
 Testing Accuracy: 0.9239
 Testing F1 Score: 0.9215

For both models, the training scores are highly concentrated around 0.92, while the testing scores are more spread out, with a mean around 0.923 for accuracy and 0.922 for F1 score. The training performance suggests that the models have high stability and low bias, however, the testing performance indicates that there may be some sensitivity to the specific training data used in each fold, leading to a poor local optimum. indicates that there may be some sensitivity to the specific training data used in each fold, leading to a poor local optimum. Since the mean training scores are very close to the mean testing scores, it suggests that the models are robust and are not overfitting. The accuracy and F1 scores are also very close to each other which suggests that the models are performing well across all labels in the dataset and that the labels in the dataset are balanced. that the models are robust and are not overfitting. The accuracy and F1 scores are also very close to each other which suggests that the models are performing well across all labels in the dataset and that the labels in the dataset are balanced. Decision trees can easily split numerical features to find precise boundaries, so they are well-suited for the rice dataset, which consists of numerical attributes.

4 The Credit Approval Dataset

4.1 Neural Networks Algorithm

We tested the neural networks algorithm on the credit dataset because it presents a complex and non-linear learning problem contains both numerical and categorical features. The hyperparameters optimized included the architecture (layer sizes), regularization values λ , and learning rates α .

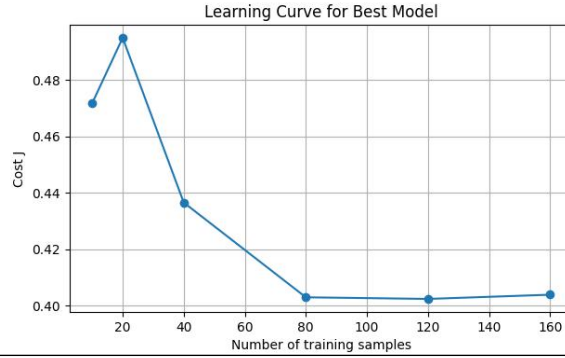
The following is a table of the performance, based on the accuracy and f1 score, of the neural network algorithm depending on the three hyperparameters.

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
((16, 8)	[46, 16, 8, 1]	0.100000	0.100000	0.874360	0.024865	0.864466	0.027051
(8,)	[46, 8, 1]	1.000000	0.050000	0.869733	0.029138	0.860984	0.030118
(4,)	[46, 4, 1]	0.100000	0.100000	0.869826	0.036683	0.860532	0.039257
(8,)	[46, 8, 1]	1.000000	0.100000	0.869850	0.025982	0.859624	0.027519
(8,)	[46, 8, 1]	0.010000	0.050000	0.869827	0.020050	0.859374	0.019345
(8,)	[46, 8, 1]	0.100000	0.050000	0.868335	0.024246	0.859090	0.023333
(16, 8)	[46, 16, 8, 1]	0.001000	0.100000	0.869861	0.029371	0.858619	0.031936
(16,)	[46, 16, 1]	0.001000	0.050000	0.868265	0.036285	0.857989	0.039753
(4,)	[46, 4, 1]	0.010000	0.050000	0.866703	0.034800	0.857677	0.036897
(4,)	[46, 4, 1]	0.001000	0.100000	0.866773	0.028553	0.857333	0.029860
(32, 16)	[46, 32, 16, 1]	0.100000	0.100000	0.868311	0.028783	0.856222	0.031245
(16,)	[46, 16, 1]	1.000000	0.050000	0.866726	0.029951	0.855795	0.034290
(8,)	[46, 8, 1]	0.100000	0.100000	0.865211	0.030648	0.855300	0.032862
(4,)	[46, 4, 1]	1.000000	0.050000	0.865164	0.033789	0.854615	0.037001
(4,)	[46, 4, 1]	1.000000	0.100000	0.863696	0.030477	0.854332	0.030515
(16, 8)	[46, 16, 8, 1]	0.010000	0.100000	0.863696	0.031244	0.853229	0.032249
(8,)	[46, 8, 1]	0.010000	0.100000	0.863696	0.032724	0.852957	0.035268
(32, 16)	[46, 32, 16, 1]	0.001000	0.100000	0.865200	0.025732	0.852798	0.028426
(16,)	[46, 16, 1]	0.010000	0.050000	0.860549	0.033442	0.852373	0.035692
(16,)	[46, 16, 1]	0.100000	0.100000	0.862146	0.031216	0.851058	0.033653
(4,)	[46, 4, 1]	0.010000	0.100000	0.860596	0.028250	0.849875	0.028500
(8,)	[46, 8, 1]	0.001000	0.100000	0.860607	0.028221	0.849765	0.029272
(16,)	[46, 16, 1]	1.000000	0.100000	0.860607	0.037888	0.849584	0.041353
(8,)	[46, 8, 1]	0.001000	0.050000	0.860596	0.021675	0.849486	0.023622
(16,)	[46, 16, 1]	0.010000	0.100000	0.862158	0.031935	0.849466	0.035368
(4,)	[46, 4, 1]	0.001000	0.050000	0.859104	0.033189	0.848866	0.035131
(32, 16)	[46, 32, 16, 1]	1.000000	0.100000	0.862204	0.032427	0.848845	0.034397
(4,)	[46, 4, 1]	0.100000	0.050000	0.857519	0.020179	0.848392	0.019781
(16,)	[46, 16, 1]	0.100000	0.050000	0.860631	0.024001	0.848242	0.024662
(32, 16)	[46, 32, 16, 1]	0.010000	0.100000	0.859069	0.026167	0.847949	0.028956
(32, 16)	[46, 32, 16, 1]	0.001000	0.050000	0.863719	0.033316	0.847370	0.037587
(16, 8)	[46, 16, 8, 1]	1.000000	0.100000	0.865339	0.038525	0.847366	0.046271
(16,)	[46, 16, 1]	0.001000	0.100000	0.857542	0.032971	0.845703	0.036293
(32, 16)	[46, 32, 16, 1]	0.100000	0.050000	0.862274	0.037436	0.842808	0.042588
(32, 16)	[46, 32, 16, 1]	0.010000	0.050000	0.854454	0.023163	0.835254	0.026117
(16, 8)	[46, 16, 8, 1]	0.010000	0.050000	0.852903	0.029232	0.831637	0.037315
(16, 8)	[46, 16, 8, 1]	0.001000	0.050000	0.851411	0.032550	0.830260	0.041729
(32, 16)	[46, 32, 16, 1]	1.000000	0.050000	0.852973	0.030127	0.825191	0.043194
(8,)	[46, 8, 1]	0.001000	0.010000	0.831703	0.029492	0.805840	0.032825
(16,)	[46, 16, 1]	0.010000	0.010000	0.829978	0.018037	0.803197	0.026804
(16,)	[46, 16, 1]	1.000000	0.010000	0.825375	0.023833	0.793457	0.032039
(8,)	[46, 8, 1]	0.010000	0.010000	0.813149	0.027031	0.771602	0.035903
(8,)	[46, 8, 1]	0.100000	0.010000	0.807216	0.041846	0.754838	0.068968
(16,)	[46, 16, 1]	0.001000	0.010000	0.788709	0.035526	0.752168	0.045184

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
(16,)	[46, 16, 1]	0.100000	0.010000	0.800829	0.045851	0.747940	0.075548
(32, 16, 8)	[46, 32, 16, 8, 1]	0.010000	0.100000	0.807321	0.062439	0.743768	0.109703
(16, 8)	[46, 16, 8, 1]	0.100000	0.050000	0.801017	0.037337	0.742917	0.077879
(16, 8)	[46, 16, 8, 1]	1.000000	0.050000	0.793232	0.058300	0.710033	0.118600
(8,)	[46, 8, 1]	1.000000	0.010000	0.767439	0.043856	0.708954	0.071521
(4,)	[46, 4, 1]	0.001000	0.010000	0.756507	0.037417	0.690657	0.079021
(4,)	[46, 4, 1]	0.010000	0.010000	0.749026	0.085388	0.640864	0.231854
(32, 16)	[46, 32, 16, 1]	1.000000	0.001000	0.502536	0.091714	0.636383	0.032655
(32, 16, 8)	[46, 32, 16, 8, 1]	0.100000	0.001000	0.453293	0.001481	0.623814	0.001404
(4,)	[46, 4, 1]	0.001000	0.001000	0.517582	0.086590	0.612690	0.033519
(4,)	[46, 4, 1]	1.000000	0.010000	0.739326	0.081586	0.601945	0.213688
(4,)	[46, 4, 1]	0.100000	0.010000	0.737089	0.107836	0.592494	0.302751
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.010000	0.001000	0.473140	0.038240	0.499603	0.249802
(16,)	[46, 16, 1]	0.010000	0.001000	0.606366	0.081820	0.493041	0.234683
(8,)	[46, 8, 1]	0.100000	0.001000	0.551473	0.077227	0.486921	0.192694
(32, 16, 8)	[46, 32, 16, 8, 1]	0.001000	0.100000	0.693440	0.107779	0.453931	0.305549
(16,)	[46, 16, 1]	0.001000	0.001000	0.557640	0.041622	0.415420	0.068093
(4,)	[46, 4, 1]	0.010000	0.001000	0.498038	0.098400	0.415149	0.176832
(8,)	[46, 8, 1]	0.001000	0.001000	0.540448	0.072059	0.389584	0.220957
(4,)	[46, 4, 1]	0.100000	0.001000	0.549807	0.069916	0.376379	0.282175
(16, 8)	[46, 16, 8, 1]	0.001000	0.001000	0.491602	0.045970	0.374735	0.305970
(16, 8)	[46, 16, 8, 1]	0.100000	0.001000	0.491602	0.045970	0.374735	0.305970
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	1.000000	0.001000	0.491602	0.045970	0.374735	0.305970
(32, 16, 8)	[46, 32, 16, 8, 1]	0.010000	0.001000	0.489936	0.045634	0.373946	0.305328
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.001000	0.001000	0.489936	0.045634	0.373946	0.305328
(32, 16)	[46, 32, 16, 1]	0.001000	0.001000	0.488421	0.040780	0.373809	0.305216
(8,)	[46, 8, 1]	0.010000	0.001000	0.502466	0.048735	0.362103	0.296180
(16,)	[46, 16, 1]	1.000000	0.001000	0.525215	0.022120	0.331294	0.277656
(32, 16)	[46, 32, 16, 1]	0.010000	0.001000	0.482185	0.033761	0.330955	0.281624
(16,)	[46, 16, 1]	0.100000	0.001000	0.536031	0.068838	0.323615	0.233016
(32, 16)	[46, 32, 16, 1]	0.001000	0.010000	0.629352	0.055012	0.300875	0.186930
(8,)	[46, 8, 1]	1.000000	0.001000	0.598501	0.050309	0.286491	0.268452
(32, 16, 8)	[46, 32, 16, 8, 1]	0.100000	0.100000	0.626357	0.084772	0.278622	0.254079
(32, 16, 8)	[46, 32, 16, 8, 1]	0.001000	0.001000	0.510064	0.045634	0.249868	0.306024
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.100000	0.001000	0.522091	0.036290	0.245622	0.300895
(16, 8)	[46, 16, 8, 1]	0.010000	0.001000	0.532943	0.042057	0.243854	0.285863
(4,)	[46, 4, 1]	1.000000	0.001000	0.545168	0.056014	0.185091	0.194691
(32, 16)	[46, 32, 16, 1]	0.100000	0.001000	0.528525	0.037014	0.125000	0.250000
(32, 16)	[46, 32, 16, 1]	0.010000	0.010000	0.578980	0.049234	0.118646	0.172696
(16, 8)	[46, 16, 8, 1]	0.001000	0.010000	0.574258	0.037998	0.104529	0.136905
(16, 8)	[46, 16, 8, 1]	0.100000	0.010000	0.565157	0.019635	0.097644	0.110062
(32, 16)	[46, 32, 16, 1]	0.100000	0.010000	0.568234	0.023305	0.095018	0.105994
(16, 8)	[46, 16, 8, 1]	1.000000	0.010000	0.568245	0.035580	0.080720	0.131137
(16, 8)	[46, 16, 8, 1]	1.000000	0.001000	0.494399	0.104902	0.078431	0.156863
(32, 16)	[46, 32, 16, 1]	1.000000	0.010000	0.559003	0.020965	0.061591	0.078798
(16, 8)	[46, 16, 8, 1]	0.010000	0.010000	0.555867	0.019778	0.048348	0.081301
(32, 16, 8)	[46, 32, 16, 8, 1]	0.010000	0.050000	0.555938	0.011978	0.038512	0.051008
(32, 16, 8)	[46, 32, 16, 8, 1]	0.001000	0.050000	0.548222	0.002753	0.006557	0.013115
(32, 16, 8)	[46, 32, 16, 8, 1]	0.001000	0.010000	0.546707	0.001481	0.000000	0.000000
(32, 16, 8)	[46, 32, 16, 8, 1]	0.010000	0.010000	0.546707	0.001481	0.000000	0.000000
(32, 16, 8)	[46, 32, 16, 8, 1]	0.100000	0.010000	0.546707	0.001481	0.000000	0.000000
(32, 16, 8)	[46, 32, 16, 8, 1]	0.100000	0.050000	0.546707	0.001481	0.000000	0.000000
(32, 16, 8)	[46, 32, 16, 8, 1]	1.000000	0.001000	0.546707	0.001481	0.000000	0.000000
(32, 16, 8)	[46, 32, 16, 8, 1]	1.000000	0.010000	0.546707	0.001481	0.000000	0.000000

architecture	layer_sizes	lambda	alpha	accuracy_mean	accuracy_std	f1_mean	f1_std
(32, 16, 8)	[46, 32, 16, 8, 1]	1.000000	0.050000	0.546707	0.001481	0.000000	0.000000
(32, 16, 8)	[46, 32, 16, 8, 1]	1.000000	0.100000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.001000	0.010000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.001000	0.050000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.001000	0.100000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.010000	0.010000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.010000	0.050000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.010000	0.100000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.100000	0.010000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.100000	0.050000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	0.100000	0.100000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	1.000000	0.010000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	1.000000	0.050000	0.546707	0.001481	0.000000	0.000000
(64, 32, 16, 8)	[46, 64, 32, 16, 8, 1]	1.000000	0.100000	0.546707	0.001481	0.000000	0.000000

The best performance is only around 87.4% accuracy and 86.4% F1, which is noticeably worse than other datasets tested in this project. The best result comes from an architecture of [46, 16, 8, 1], $\lambda = 0.1$ and $\alpha = 0.1$. A prominent pattern here is there seems to be a mid-way point where larger networks begin to hurt the model's performance as opposed to helping. Below is a learning curve showing the value of J as a function of the number of training instances presented to the above best network. As evidenced, the cost decreases as a function of the number of training instances, which is as expected and proves that the neural network algorithm is able to learn the credits dataset.



4.2 KNN

k	Accuracy	F1 Score
1	0.806250	0.780158
3	0.859375	0.845093
5	0.864062	0.853219
7	0.873437	0.864596
9	0.862500	0.854968
11	0.865625	0.858536
13	0.856250	0.848834
15	0.860938	0.854310
17	0.862500	0.858192
19	0.864062	0.860247
21	0.865625	0.860856
23	0.857812	0.853120
25	0.864062	0.858874
27	0.867188	0.863106
29	0.857812	0.854402
31	0.860938	0.856263
33	0.868750	0.865578
35	0.857812	0.852417
37	0.862500	0.857252
39	0.864062	0.856573
41	0.860938	0.853287
43	0.856250	0.848844
45	0.862500	0.855567
47	0.860938	0.854175
49	0.864062	0.856989
51	0.856250	0.849728



Figure 10: Credit Approval Accuracy and F1 Score Distribution

Similar to the rice data set, the performance increases after $k=1$ (though more sharply for this dataset) and begins a relative plateau in performance. However, there are slightly more fluctuations in this plateau with an average fluctuation of about $\pm 1\%$. The best performance is at $k = 7$ at 87% accuracy and 86% F1 score. We

chose KNN for this dataset because it is robust on categorical and numerical data, and it is a relatively small dataset meaning that it will run relatively fast for KNN.

4.3 Decision Trees Algorithm

We chose to test decision trees on the credit approval dataset because it has a moderate number of features (15) and a relatively small number of samples (653), which allows us to test two different hyperparameters; tree depths and minimum split sizes. This is shown in the tuning results below where we tested 6 different tree depths and 4 different minimum split sizes. We chose to test decision trees on the credit approval dataset because it has a moderate number of features (15) and a relatively small number of samples (653), which allows us to test two different hyperparameters; tree depths and minimum split sizes. This is shown in the tuning results below where we tested 6 different tree depths and 4 different minimum split sizes.

Tuning for dataset with 653 rows and 15 features.

Testing Depths: [2, 4, 6, 8, 10, 12]

Testing Splits: [32, 65, 97, 130]

Depth	Split	Test Acc	Test F1
2	32	0.8622	0.8621
2	65	0.8639	0.8636
2	97	0.8622	0.8622
2	130	0.8622	0.8620
4	32	0.8378	0.8358
4	65	0.8482	0.8473
4	97	0.8588	0.8580
4	130	0.8577	0.8566
6	32	0.8318	0.8293
6	65	0.8302	0.8288
6	97	0.8501	0.8491
6	130	0.8610	0.8606
8	32	0.8378	0.8356
8	65	0.8576	0.8568
8	97	0.8496	0.8489
8	130	0.8620	0.8612
10	32	0.8377	0.8356
10	65	0.8409	0.8397
10	97	0.8437	0.8423
10	130	0.8504	0.8495
12	32	0.8375	0.8350
12	65	0.8451	0.8439
12	97	0.8668	0.8654
12	130	0.8635	0.8626

```
=====
BEST CONFIGURATION:
Max Depth: 12
Min Split: 97
F1 Score: 0.8654
=====
```

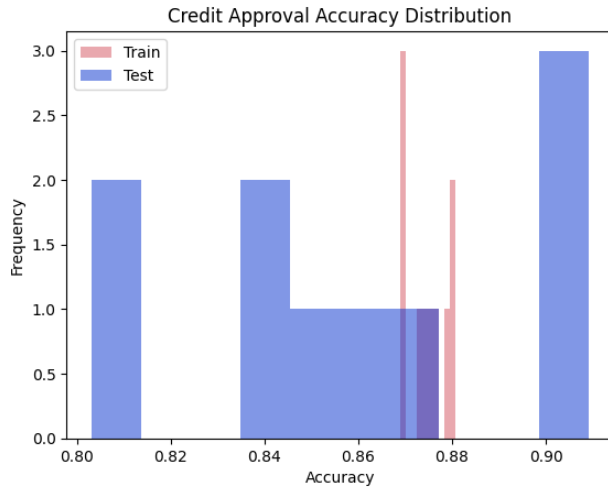


Figure 11: Credit Approval Accuracy Distribution



Figure 12: Credit Approval F1 Score Distribution

Final Results:

Training Accuracy: 0.8746
 Training F1 Score: 0.8741
 Testing Accuracy: 0.8606
 Testing F1 Score: 0.8601

The test scores are widely spread which suggests the model is sensitive to the specific training data used in each fold, leading to a poor local optimum. Especially in smaller datasets, the presence/absence of a few instances in a training fold can have a significant impact on the tree structure and thus the performance. The training scores are more concentrated around 0.87, while some test folds outperform the training performance. This suggests that the model is stable during training, but is not consistent on generalizing depending on the test fold. The accuracy and F1 scores are also very close to each other which suggests that the model is performing well across all labels in the dataset and that the labels in the dataset are balanced. Since the credit approval dataset has some categorical features, they need to be factorized. If there are a lot of unique values in a categorical feature, it can lead to a large number of splits in the decision tree, which may be why there is more spread in the test scores for this dataset.

Final Results:

Training Accuracy: 0.8746
 Training F1 Score: 0.8741
 Testing Accuracy: 0.8606
 Testing F1 Score: 0.8601

The test scores are widely spread which suggests the model is sensitive to the specific training data used in each fold, leading to a poor local optimum. Especially in smaller datasets, the presence/absence of a few instances in a training fold can have a significant impact on the tree structure and thus the performance. The training scores are more concentrated around 0.87, while some test folds outperform the training performance. This suggests that the model is stable during training, but is not consistent on generalizing depending on the test fold. The accuracy and F1 scores are also very close to each other which suggests that the model is performing well across all labels in the dataset and that the labels in the dataset are balanced. Since the credit approval dataset has some categorical features, they need to be factorized. If there are a lot of unique values in a categorical feature, it can lead to a large number of splits in the decision tree, which may be why there is more spread in the test scores for this dataset.

	Handwritten Digits		Parkinson's		Rice Grains		Credit Approval	
	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
Neural Networks	0.936010	0.936004			0.928871	0.937713	0.874360	0.864466
K-Nearest Neighbors	0.97109	0.97281	0.866239	0.92118	0.9237	0.91048	0.86009	0.85289
Decision Trees					0.9239	0.9215	0.8606	0.8601
Random Forest	0.989267	0.98767	0.82698	0.75305				

Extra Credit

Describe what you did as part of that task and present the corresponding results. Any source code you develop should be included in your Gradescope submission.

[QE.1] [Extra Credit Question #1] (10 Points)

You may earn up to 10 extra points if you analyze all four datasets using an additional algorithm. In other words, you would be evaluating more than two algorithms on each of the datasets discussed in Section 1.

Solution. We implemented this extra credit question. This is evidenced by there being three algorithms tested on each dataset.